BCS VII SEMESTER • CLOUD COMPUTING CAPSTONE PROJECT SPECIFICATION

RescueGrid: System Objectives, Mathematical Formulations & Implementation Architecture

A Distributed, Real-Time Cloud Platform for Zero-Waste Food Redistribution & Cryptographic Chain-of-Custody

Project Domain: Cloud Computing & Geospatial Distributed Systems	**Architecture:** 3x Stateless API Replicas behind Nginx Load Balancer	**Verification:** 100% Passed (4 Test Suites, 53+ Unit/Integration Specs)
Data Layer: Redis 7 GEO + PostgreSQL 15 ACID Ledger	**Frontend:** React 18, Leaflet Radar, OpenStreetMap, Dual Theme	**Handover Security:** HMAC-SHA256 Non-Repudiation + 6-Digit Gate OTP

1. Core Objectives & Problem Statement

The Urban Food Paradox: Modern urban regions face a stark paradox. Commercial food producers (hotels, banquets, university cafeterias, and caterers) routinely prepare surplus meals that are discarded at night. Simultaneously, verified hunger-relief shelters and community kitchens within a 5–15 km radius experience chronic food shortages. This failure is fundamentally logistical, temporal, and computational.

- **Perishability & The 3-Hour Microbial Window:** Prepared hot food rapidly enters the biological temperature danger zone (5°C – 60°C). Under FSSAI/FDA safety guidelines, meals must be transported, verified, and consumed within 2–4 hours. Static bulletin boards or asynchronous WhatsApp channels cannot compute route ETAs or deadline feasibility.
- **Distributed Double-Claiming Race Condition:** When an urgent surplus notification is published publicly, multiple shelters dispatch vehicles concurrently. Without atomic distributed locks across cloud instances, multiple drivers arrive at the kitchen gate for the same batch, causing wasted fuel, duplicate trips, and operational disputes.
- **Repudiation & Chain-of-Custody Failure:** Donors face legal and brand liabilities regarding food tampering and theft. Informal handovers lack verifiable cryptographic audit trails proving that the authorized shelter courier collected the exact batch.

RescueGrid Core Objective: To engineer a **high-throughput, horizontally scalable cloud platform** that automates real-time geospatial proximity matching, enforces **zero double-claims via atomic Redis 7 Lua check-and-set operations**, guarantees **microbial deadline feasibility**, and certifies legal handovers via **HMAC-SHA256 signatures and 6-digit gate OTP codes**.

2. Mathematical Formulations & Spatial Computations

Every dispatch, perimeter visualization, and scoring calculation in RescueGrid is governed by explicit mathematical proofs:

2.1 Geodesic Distance Computation (Haversine Formula)

Given donor coordinates (ϕ_1, λ_1) and recipient coordinates (ϕ_2, λ_2) in radians, the great-circle surface distance d over a spherical earth of mean radius R = 6371.0088 km is calculated as:

Haversine Equation:

$$\Delta\phi = \phi_2 - \phi_1, \quad \Delta\lambda = \lambda_2 - \lambda_1$$

$$a = \sin^2(\Delta\phi / 2) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2(\Delta\lambda / 2)$$

$$c = 2 \cdot \operatorname{atan2}(\sqrt{a}, \sqrt{(1 - a)})$$

$$d = R \cdot c \quad \text{[where } R = 6371.0088 \text{ km]}$$

2.2 Dynamic Radar Viewport Bounding Box Trigonometry

When a shelter sets a scanning radius r (km), the Leaflet radar must automatically zoom out and frame the circular scan perimeter with buffer β = 1.25 (25% margin). Because 1° of latitude is constant (~111.0 km) while 1° of longitude contracts with the cosine of center latitude ϕ_c:

Viewport Bounding Box Derivation:

$$r_{eff} = \beta \cdot r = 1.25 \cdot r$$

$$\Delta\phi = r_{eff} / 111.0 \quad \text{(degrees latitude)}$$

$$\Delta\lambda = r_{eff} / [\, 111.0 \cdot \cos(\phi_c \cdot \pi / 180) \,] \quad \text{(degrees longitude)}$$

$$\text{SouthWest} = (\phi_c - \Delta\phi, \lambda_c - \Delta\lambda), \quad \text{NorthEast} = (\phi_c + \Delta\phi, \lambda_c + \Delta\lambda)$$

Leaflet Execution: `map.fitBounds([[SW], [NE]], { padding: [25, 25], maxZoom: 15, animate: true })`

2.3 Urban Transit Velocity Model & Total Mission Duration

Metropolitan courier dispatches operate under non-linear traffic friction. The model incorporates average urban velocity v_{urban} = 25 km/h, congestion multiplier $\kappa_{traffic}$ = 1.2, donor loading buffer τ_{load} = 15 min, and shelter unloading buffer τ_{unload} = 15 min:

$$\textbf{Transit Duration Model:}$$
$$t_{one\text{-}way} = (d \,/\, v_{urban}) \cdot \kappa_{traffic} \cdot 60 \quad [\text{minutes}]$$
$$t_{round\text{-}trip} = 2 \cdot t_{one\text{-}way}$$
$$T_{mission} = t_{round\text{-}trip} + \tau_{load} + \tau_{unload} = 2 \cdot t_{one\text{-}way} + 30 \text{ minutes}$$

2.4 Hard Feasibility Filtering & Deadline Slack Formulations

A match candidate s is rejected outright unless all hard physical constraints are satisfied. Let t_{claim} be the claim timestamp, T_{pickup} be the donor's pickup deadline, t_0 be cooking completion, and W_{safe} be the biological safety window (e.g. 180 min):

$$\textbf{Operational Slack Equations \& Feasibility Conditions:}$$
$$t_{arrive_donor} = t_{claim} + t_{one\text{-}way}$$
$$t_{arrive_shelter} = t_{arrive_donor} + \tau_{load} + t_{one\text{-}way}$$
$$T_{expire} = t_0 + W_{safe}$$
$$Slack_{pickup} = T_{pickup} - t_{arrive_donor} \geq 0$$
$$Slack_{consumption} = T_{expire} - (t_{arrive_shelter} + \tau_{unload}) \geq 0$$
$$\text{Hard Constraint Gate: Feasible}(s, b) \Leftrightarrow (Slack_{pickup} \geq 0) \wedge (Slack_{consumption} \geq 0) \wedge (C_s \geq Q_b) \wedge (\text{DietaryCompatible}(s, b) == 1) \wedge (\text{Verified}(s) == 1)$$

2.5 Multi-Criteria Optimization Scoring Function (MAUT)

All candidates satisfying hard feasibility are ranked using normalized Multi-Attribute Utility Theory (MAUT). The composite score $S(s, b) \in [0, 1]$ combines four independent dimensions with normalized weights $\sum w_i = 1.0$:

Score Dimension	Mathematical Definition	Weight	Objective Rationale
Proximity Fit (S_{dist})	$S_{dist} = 1 - (d / r_{max})$	**0.35** (35%)	Minimizes courier vehicle emissions, fuel, and transit risk.
Capacity Match (S_{cap})	$S_{cap} = 1 - (\lvert C_s - Q_b \rvert / (C_s + Q_b))$	**0.25** (25%)	Prevents overwhelming small shelters or underutilizing large ones.
Deadline Urgency (S_{urg})	$S_{urg} = 1 - (Slack_{consumption} / W_{safe})$	**0.25** (25%)	Prioritizes meals with tighter margins before biological spoilage.
Active Demand (S_{dem})	S_{dem} = 1.0 if category matches else 0.0	**0.15** (15%)	Directly fulfills declared dietary needs from recipient shelters.

Deterministic Tie-Breaking: When scores tie ($|S_1 - S_2| < 10^{-5}$), tie-breaking is resolved deterministically via alphabetical comparison of recipient UUIDs: TieBreak(s_1, s_2) = strcmp(s_1.id, s_2.id). This guarantees identical, idempotent recommendations across all stateless cloud replicas.

2.6 Cryptographic Non-Repudiation & Dual-Signature Handover

To eliminate disputes and establish a tamper-proof chain of custody, RescueGrid implements a **dual-token cryptographic gate**. A 64-character HMAC-SHA256 signature is paired with a 6-digit zero-padded OTP code:

$$\textbf{Token Generation \& Verification Equations:}$$
$$Token_{raw} = \text{ListingUUID} \,||\, \text{RecipientOrgUUID} \,||\, \text{DonorOrgUUID} \,||\, t_{claim} \,||\, \text{Nonce}$$
$$\text{Signature} = \text{HMAC-SHA256}(K_{master}, \text{Token}_{raw}) \quad \text{(256-bit tamper-proof proof)}$$
$$\text{OTP} = [\ \text{CRC32(Signature) mod } 10^6\ ] \quad \text{formatted as '\%06d' (6-digit physical kitchen gate code)}$$
$$\textbf{Timing-Safe Constant-Time Verification:}$$
$$\text{Verify}(A, B) = [\ \sum_{i=0}^{n-1} (A[i] \oplus B[i]) == 0\ ] \quad \text{(prevents timing side-channel attacks)}$$

2.7 Distributed Atomic Claiming Engine (Redis Lua Script)

To eliminate race conditions when dozens of shelters claim the same surplus batch concurrently across 3 API replicas, the claim operation is executed as a non-preemptible **Redis Lua script** running single-threaded inside the Redis engine:

```
Atomic Check-and-Set State Machine:
local current_status = redis.call('HGET', KEYS[1], 'status')
if current_status == 'AVAILABLE' then
  redis.call('HSET', KEYS[1], 'status', 'CLAIMED', 'claimed_by', ARGV[1], 'claimed_at', ARGV[2])
  redis.call('ZREM', 'geo:donations', ARGV[3]) -- Instant spatial eviction
  return { 1, 'CLAIM_SUCCESS', ARGV[1] }
else return { 0, 'ERR_ALREADY_CLAIMED', current_status } end
```

3. Full System Architecture & Implemented Components

RescueGrid is completely containerized, deployed, and operational using 6 Docker containers orchestrated via Docker Compose:

Layer / Service	Container / Tech	Implemented Capabilities & Operational Status
Load Balancer	rescuegrid_lb Nginx 1.25 Alpine	Exposes port 8080. Round-robin proxying across 3 API replicas. WebSocket upgrade pass-through with persistent HTTP keep-alive.
Stateless API Cluster	rescuegrid_api_1 rescuegrid_api_2 rescuegrid_api_3 Node.js 18 + Express	Horizontally scaled replicas. Zero local in-memory shared state. All state delegated to Redis & Postgres. Shared JWT secret and multi-replica session synchronization.
In-Memory Geospatial State	rescuegrid_redis Redis 7.2 Alpine	Dual Geospatial sorted sets (geo:donations, geo:shelters). Atomic Lua claiming engine. Keyspace notifications (KEx) for expired food batches.
ACID Audit Ledger	rescuegrid_postgres PostgreSQL 15 Alpine	Relational persistence with foreign keys, transactional migrations, and immutable audit_ledger recording all lifecycle events and timestamps.
Frontend Web Application	client React 18 + Leaflet + Shadcn	Interactive Radar with dynamic blue scan circle, OpenStreetMap zero-key tiles, dual theme (Light/Dark mode with toggle), Hub-and-Spoke courier vectors, and Apple Wallet OTP pass.

3.2 Implemented Frontend Portals & UI Workflows

- **Food Producer Portal (DonorPage.js):** Structured surplus broadcast form with quantity, food type (Cooked/Raw, Veg/Non-Veg), prep timestamp, shelf-life countdown clock, and an active donation handover management gate with real-time OTP verification terminal.
- **Inbound Proximity Food Radar (ShelterRadarMap.js):** Real-time recipient scanning radar. Features an interactive radius slider (1–50 km) that dynamically projects a **blue dashed circular perimeter**, renders available surplus batches as emerald pins, and automatically fits viewport coordinates.
- **Hub-and-Spoke Multi-Delivery Route Lines:** When a shelter claims one or multiple simultaneous food donations, vibrant cyberpunk-colored vector lines (#38bdf8 Sky Blue, #34d399 Emerald, #fbbf24 Amber, #c084fc Purple) connect each donor kitchen directly to the shelter hub.
- **Digital Apple Wallet Pass & OTP Modal:** Claiming a donation instantly displays a clean digital pass containing the 6-digit OTP code, cryptographic verification token, pickup address, and driver instructions.
- **Dual-Theme Architecture (Light Mode & Dark Mode):** Complete Shadcn UI styling with an instant header toggle button (■■ Light / ■ Dark), full localStorage persistence, and client-side dark matter filter for night-mode OpenStreetMap tiles with zero external API dependencies.

3.3 Verification Ledger & Comprehensive Automated Test Suite

Every mathematical formula, constraint filter, and cloud component is backed by an automated verification suite:

Verification Test Suite	Command	Tests Passed	Key Invariants Validated
Core Unit & Algorithms	npm run test:units	**25 / 25 PASS (100%)**	Haversine spherical accuracy (±0.01 km), coordinate bounds validation, HMAC-SHA256 non-repudiation, constant-time verification, Redis Lua script syntax.
Deadline Matching Engine	npm run test:matching	**28 / 28 PASS (100%)**	Round-trip travel times, negative consumption slack hard rejection, dietary mismatch rejection, capacity bounds, deterministic tie-breaking.
Distributed Concurrency	npm run test:concurrency	**PASS (Race-Free)**	50 parallel claims dispatched to 3 API replicas simultaneously. Exactly 1 claim succeeded with lock; 49 safely received ERR_ALREADY_CLAIMED.
Keyspace TTL Sweeper	npm run test:ttl	**PASS (Zero Stale Data)**	Expired food batches automatically evicted from Redis geospatial index and marked EXPIRED in PostgreSQL audit ledger.
Client Production Build	npm run build	**PASS (0 Warnings)**	Compiled React production bundle with minified CSS tokens, asset tree optimization, and zero ESLint errors.

4. System Comparison: Traditional Portals vs. RescueGrid

The following matrix summarizes the architectural advantages of the RescueGrid implementation over traditional food rescue approaches:

Feature / Requirement	Traditional Food Rescue Apps	RescueGrid Cloud Platform
Claiming Mechanism	Asynchronous bulletin board (high race conditions)	**Sub-millisecond Atomic Lua Check-and-Set Lock**
Geospatial Search	Periodic SQL queries (high disk I/O latency)	**In-Memory Redis 7 GEOSEARCH (sub-ms index)**
Perishability Protection	Manual human inspection (food frequently spoils)	**Mathematical Slack & Road Transit Velocity Filter**
Handover Verification	Unsigned paper forms or verbal confirmation	**HMAC-SHA256 Signature + 6-Digit Gate OTP Pass**
Cloud Scalability	Single monolithic server (single point of failure)	**Nginx Load-Balanced 3x Stateless Cluster + Docker**
Visual Topology	Static text tables or basic single pin map	**Dynamic Radar Perimeter + Hub-and-Spoke Polylines**

Conclusion & Engineering Sign-Off: RescueGrid demonstrates that cloud computing paradigms—stateless horizontal scaling, in-memory distributed geospatial indexes, non-preemptible atomic scripts, and cryptographic tokenization—can completely eliminate urban food redistribution failure modes. All components are containerized, mathematically validated, and fully operational.